

Agent Loop 三方对比

同一个「模型 ↔ 工具反复交互」内核，三种工程取舍：从一个 `for` 循环到一千七百行的状态机

对象： `claudecode` (泄露源码) · `openclaw-2026.3.23` (内核代号 Pi) · `ai/` (本项目) · 生成日期：2026-06-23

- Claude Code — 软件工程 CLI, `claudecode/query.ts` (~1730 行单循环)
- OpenClaw — 多渠道私人助理, `src/agents/pi-embedded-runner/run.ts` (~1716 行编排壳)
- ai — 你的本地编码 agent, `ai/src/agent/engine.ts` (85 行)

一句话结论

Claude Code

把循环本身做成产品。

一个 `while(true)` 生成器里塞进了全部上下文工程与容错：多级压缩、模型回退、流式工具并发、对超长/超 token 的逐级恢复。终止有七八种 `reason`。

OpenClaw

自己不写循环，而是包住一个循环。

真正的「模型↔工具」迭代交给内嵌引擎 `@mariozechner/pi-coding-agent`；`run.ts` 是它外面的调度壳：会话队列、密钥轮换、模型回退、溢出补偿、多聊天渠道。

ai (本项目)

把循环还原成它最朴素的样子。

一个 `for (step < 25)` 跑完：非流式补全 → 没有工具调用就结束 → 否则顺序执行工具回灌结果。上下文靠「丢最旧的几条」维持，干净、好读、够用。

同源不同量。 三者内核完全一致——发消息 → 模型要么回话要么调工具 → 执行工具 → 结果塞回 → 再发。差异全在「循环外面包了多少层」：Claude Code 把容错与上下文压缩长进了循环里；OpenClaw 把这层职责外包给 Pi 引擎、自己专注做多渠道网关；ai 则刻意不包，换来一眼读懂的 85 行。

核心循环（伪代码并排）

`ai` · 一个 `for` 循环，全部真相都在这 85 行里

```
for (let step = 0; step < maxSteps; step++) { // 默认 25 步封顶
  const { content, toolCalls } = await chatComplete(history, { tools }) // 非流式一轮补全
  history.push({ role:'assistant', content, tool_calls })
  if (content) yield { type:'text', content }

  if (!toolCalls.length) return // ← 没有工具调用 = 最终答复，结束

  for (const tc of toolCalls) { // 顺序执行（非并发）
    try { result = await runTool(tc.name, args) }
    catch (e) { result = '错误: ' + e.message } // 工具异常 → 字符串回灌，不中断
```

```

    history.push({ role:'tool', tool_call_id: tc.id, content: result })
  }
}
yield { type:'text', content: `[已达最大步数 ${maxSteps}, 停止]` } // 兜底
ai/src/agent/engine.ts:45

```

Claude Code · 同一个循环，外加每轮的上下文工程与逐级恢复

```

while (true) { // queryLoop — 单一长循环，靠 state 在迭代间传递
  messages = applyToolResultBudget(messages) // ① 工具结果按 tool_use_id 限额裁剪
  messages = snipCompact(messages) // ② 轻量删减
  messages = await microcompact(messages) // ③ 微压缩 (编辑缓存，不重发)
  messages = applyCollapses(messages) // ④ context-collapse 投影视图
  ({ compactionResult } = await autocompact(messages)) // ⑤ 模型总结式压缩
  if (atBlockingLimit()) return {reason:'blocking_limit'}

  for await (msg of callModel({messages, tools, fallbackModel})) {
    withhold(可恢复错误) // 暂扣 prompt-too-long / max-output-tokens
    streamingToolExecutor.addTool(toolBlock) // ⑥ 工具在「流式过程中」就开跑
  } // FallbackTriggeredError → 换模型重试整轮

  if (!needsFollowUp) { // 模型没要工具 → 准备收尾，但先过一串恢复闸
    if (withheld413) { collapseDrain() ?? reactiveCompact(); continue }
    if (maxOutputTok) { 8k-64k escalate ?? 注入恢复消息; continue }
    if (stopHook.blocks) { continue } // stop-hook 可挡住收尾
    if (tokenBudget.continue) { continue } // +500k 自动续跑
    return {reason:'completed'}
  }

  for await (r of runTools(blocks, canUseTool)) yield r // 权限闸 canUseTool 逐个放行
}
claudecode/query.ts:241 (queryLoop)

```

OpenClaw · 循环交给 Pi 引擎，自己在外套「重试 / 回退 / 队列」

```

enqueueSession(() => enqueueGlobal(async () => { // 会话级 + 全局级两道串行队列
  while (true) { // run.ts:885 — 这是「尝试」循环，不是 agent 循环
    attempt = await runEmbeddedPiSession({ // ← 真正的 模型+工具 循环在 Pi 引擎内部
      model, authProfile, abortSignal, // (流式、工具执行、auto-compaction 都由 Pi 负责)
    })
    if (attempt.aborted) break
    if (contextOverflow(attempt)) { // 上下文溢出 → 显式压缩后重试
      if (!attempt.alreadyCompacted) await compactEmbeddedPiSession()
      continue
    }
    if (authError(attempt) && nextProfile) { 轮换密钥; continue } // 多 API key 冷却轮换
    if (modelError(attempt) && fallbackConfigured) { 换模型; continue }
    break
  }
})
// 外层还有: EmbeddedBlockChunker 把流式块整理成「只发最终回复」给 Telegram/WhatsApp 等渠道
openclaw-2026.3.23/src/agents/pi-embedded-runner/run.ts:287

```

关键区别在「循环的边界画在哪」。 ai 和 Claude Code 的循环边界 = agent 循环本身；OpenClaw 的 `run.ts` 循环边界 = 整轮的重试，agent 循环被藏在 `runEmbeddedPiSession` 里。所以读 OpenClaw 源码看不到「解析 tool_calls」这种代码——那一层是 Pi 引擎的。

逐维度对照

维度	Claude Code	OpenClaw	ai (本项目)
循环载体	<code>while(true)</code> async generator, 单文件 1730 行	外层「尝试」 <code>while(true)</code> ; 真正 agent 循环在 Pi 引擎	<code>for (step<25)</code> async generator, 85 行
模型调用	流式 (SSE), <code>callModel</code> 边流边产出	流式, 由 Pi 引擎驱动; 外层只看最终态	非流式 <code>chatComplete</code> (<code>streamChat</code> 存在但循环未用)
协议	Anthropic Messages API (原生 content blocks)	多 provider (Anthropic / OpenAI / ..., 经 pi-ai 抽象)	任意 OpenAI 兼容 <code>/chat/completions</code> (默认 DeepSeek)
终止条件	无 <code>tool_use</code> + 过七八道恢复闸 → <code>reason: completed</code> , 另有 <code>blocking_limit</code> / <code>aborted</code> / <code>model_error</code> / <code>prompt_too_long</code> ...	Pi 内部正常结束; 外层在溢出/认证/模型错误时重试, 否则 <code>break</code>	无 <code>toolCalls</code> → <code>return</code> ; 或 <code>step</code> 达 25 → 兜底文字
工具执行	流式并发: <code>StreamingToolExecutor</code> 在模型仍在输出时就开跑; <code>canUseTool</code> 权限闸	由 Pi 引擎执行; <code>openclaw</code> 注入工具集 + <code>before/after-tool-call</code> 钩子	顺序 <code>for</code> 循环逐个 <code>await runTool</code> ; <code>try/catch</code> 转错误字符串回灌
上下文压缩	五级: <code>budget</code> 裁剪 / <code>snip</code> / <code>microcompact</code> / <code>context-collapse</code> / 模型总结 <code>autocompact</code>	Pi 的 <code>auto-compaction</code> (按 <code>contextTokens</code> 上限) + <code>openclaw</code> 显式 <code>compactEmbeddedPiSession</code> 重试补偿	无总结。 <code>SessionStore.trim()</code> 超 60 条就从 <code>user</code> 边界丢最旧的几条
超长 / 超 token 恢复	逐级: <code>collapse-drain</code> → <code>reactive compact</code> ; <code>max-output 8k</code> → <code>64k</code> 升档→注入恢复消息	<code>contextOverflow</code> → 压缩后整轮重试 (带 <code>diag</code> 计数防死循环)	无专门恢复; 靠 <code>trim</code> 提前防膨胀 + 25 步封顶兜底
模型回退	有: <code>FallbackTriggeredError</code> 换 <code>fallbackModel</code> 重试整轮 + 清理孤儿消息	有: 模型回退链 + 认证 <code>profile</code> 冷却轮换	无
多会话 / 并发	子 agent (<code>AgentTool</code>) 有独立 <code>queryTracking</code> 链; 主循环单会话	会话级 + 全局级命令队列 (<code>lane</code>) 串行化; 多聊天渠道并存	<code>SessionStore</code> <code>Map</code> : 私聊按用户、群按群各存一份历史
扩展 / 钩子	<code>post-sampling</code> / <code>stop</code> / <code>stop-failure</code> hooks、MCP、技能预取、记忆预取	插件系统、MCP、 <code>before/after compaction</code> & <code>tool-call</code> 钩子	<code>channel</code> 注入的 <code>extraTools</code> (如 QQ 的 <code>send_image</code>), 无钩子
权限	<code>canUseTool</code> 逐工具放行 + <code>permission mode</code> (含 <code>plan</code> 模式)	沙箱 / 工作区挂载路径限制 + 渠道白名单	无门禁——系统提示直言「你有权限操作设备, 别推辞」

维度	Claude Code	OpenClaw	ai（本项目）
内置工具	Bash/Edit/Read/Write/Glob/Grep/Agent/Web... —整套 IDE 级	编码工具 + 渠道消息工具 (sendMessage 等) + 记忆 / 搜索	write_file / read_file / list_dir / run_bash / send_email / stock_quote

复杂度量表（数量级直觉）

同一根内核，工程包裹量相差近 20 倍。下面是「循环主体代码行」的相对感受，非精确统计。



行数为对应文件粗略规模；OpenClaw 的 1716 行是「编排壳」，真正 agent 循环在外部 Pi 引擎里，故其内核复杂度被低估。

同一请求下三者怎么跑

请求：「读一下 config.ts，然后把里面的端口改成 8080」（需要：读文件 → 改文件，两轮工具）

1 chatComplete
模型回 tool_call:
read_file

2 runTool 顺序执行
读到内容，回灌为 tool 消息

3 chatComplete
模型回 tool_call:
write_file

4 runTool
写盘，回灌结果

1 压缩 5 件套 + 预取
budget/snip/micro/collapse/autocompact

2 流式 callModel
Read 工具边流边执行

3 canUseTool 放行
Edit 命中权限 → 经确认后写入

4 收尾闸
stop-hook / token-budget 检查

1 入队（会话+全局 lane）
保证同会话不并发

2 runEmbeddedPiSession
Pi 引擎内部跑完读→改两轮

3 attempt 返回
检查溢出 / 认证 / 模型错误

4 无异常 → break
否则压缩/换 key/换模型后重试

5 BlockChunker 整流

file:///Users/lteu/progetto/agent/%E5%AF%B9%E6%AF%94-agent-loop-%E4%B8%89%E6%96%B9.html

4/5

5 chatComplete
无 toolCalls → return

5 reason: completed
每轮都重跑全部压缩与恢复检查

只把最终回复推给聊天渠道

● ai: 5 步, 零额外加工

● Claude Code: 同样两轮工具, 外裹上下文工程

● OpenClaw: 循环细节被 Pi 封装, 外层管调度

对照之后，ai 可以借鉴什么

当前 85 行的取舍是对的——简单、可读、对「单可信用户」场景够用。若要往前走，按性价比排序：

改进点	现状	可借鉴谁 · 怎么做
① 流式输出	循环用非流式 chatComplete；streamChat 已写好却没接进 loop	CC 边流边产出。终端/QQ 体验立刻变好——把 chatComplete 换成流式聚合即可，工具调用仍在流末统一解析。
② 总结式压缩	只有 trim() 丢最旧消息——长对话会直接丢上下文	CC 的 autocompact / OC 的 compact：超阈值时调一次模型把旧历史总结成一段，替换原文。投入小、收益大。
③ 工具结果限额	read_file 截到 20000 字符，但多个大结果叠加仍可能撑爆	CC 的 applyToolResultBudget：对「聚合后」的工具结果设总预算，而非逐个截断。
④ 模型回退	单一 provider，请求失败直接抛错	OC 的回退链：主模型 5xx/限时自动切备用 baseUrl/model，对你的多 provider 配置天然契合。
⑤ 工具并发	多个 toolCalls 顺序 await	CC 流式并发偏复杂；轻量做法：对无副作用的只读工具（read_file/list_dir/stock_quote）用 Promise.all 并发，写类工具仍串行。
⑥ 会话串行	同一会话若并发收到两条消息，会同时改同一份 history	OC 的 lane 队列：给每个 session 加一把串行锁，避免 history 被并发追加写花。

别照抄 Claude Code 的循环。它那 1730 行里大部分是为「无人值守、超长会话、多模型、严格权限」付的税。ai 的场景（可信用户、短会话、手机端简洁回复）只需选摘①②④三项，就能在不牺牲可读性的前提下显著变稳。

依据源码：ai/src/agent/engine.ts、ai/src/agent/session.ts、ai/src/llm.ts、ai/src/tools.ts；claudecode/query.ts (queryLoop)；openclaw-2026.3.23/src/agents/pi-embedded-runner/run.ts 及 compact.ts。OpenClaw 的真正 agent 循环环位于其依赖 @mariozechner/pi-agent-core / pi-coding-agent（本仓库未安装 node_modules，故按其调用壳与文档行为描述）。